

NeuralToneTransform

本课题需利用深度学习技术（如 NAM或 LSTM）对乐器的非线性音色特征进行‘黑盒建模’，通过采集高质量的配对音频数据训练模型。

通过加载训练好的深度学习模型，实现将输入的普通乐器信号瞬间转化为具备极高保真度的目标乐器音色（如提前训练好的昂贵的管弦乐器、经典电子合成器或特定音箱模拟），且在保证极低演奏延迟的前提下，能够精准还原原信号的动态表情与演奏细节。

参考

1. NAM 官方开源库 (作者: Steven Atkinson)

<https://www.neuralampmodeler.com/>

NAM 的整个生态（从训练到部署）都是在 GitHub 上公开的

- **Python 训练端 (`neural-amp-modeler`)**: 包含其核心算法的 PyTorch 实现。主要使用了 **WaveNet** 风格的 **TCN**（时域卷积网络）以及 **LSTM**。你可以直接看到它是如何进行数据预处理（ESR 损失函数）和网络构建的。

2. GuitarML 与 Proteus

这是另一个非常著名的开源吉他音色建模生态，非常适合拿来与 NAM 做对比研究：

- **核心算法**：GuitarML 早期使用了单纯的 **LSTM (GuitarLSTM)**，后来进化到使用基于 WaveNet 的架构，甚至结合了传统的 DSP 算法。
- **开源插件 Proteus**：它的源码非常清晰，对于初学 JUCE 和深度学习音频部署的学生来说，它的代码结构可能比 NAM 更容易理解。

阶段一：理论储备与基建准备

****目标**：搞清楚什么是nam，要实现什么样的效果

- **核心任务**：
 1. **文献阅读**：精读 DeepMind 的原始论文 *WaveNet: A Generative Model for Raw Audio*，理解什么是“感受野 (Receptive Field)”、“膨胀卷积 (Dilated Convolution)”和“因果性 (Causality)”。
 2. **代码环境**：配置 Python 3.9+ 环境，安装 PyTorch、Torchaudio、Librosa 和 Matplotlib。

3. **跑通 Baseline**：从 GitHub 克隆官方的 NAM 库，先用它提供的公开数据集跑通一遍训练流程，看看一个完整的 AI 炼丹炉长什么样。
- **阶段产出**：一份关于 TCN/WaveNet 处理音频信号的文献综述，以及配置好的云端或本地 GPU 训练环境。

阶段二：数据采集与亚采样级对齐

目标：在音频黑盒建模中，数据质量决定了模型的上限。

- **核心任务**：
 1. **采集干湿对 (Re-amping)**：录制 5-10 分钟极其干净的吉他/贝斯 DI 信号（干声），包括和弦、单音、扫弦、闷音等所有演奏技法。将干声输入到一台真实的音箱或失真效果器中，录制输出的声音（湿声）。
 2. **相位对齐 (生死攸关的一步)**：编写一段 Python 脚本，使用**互相关算法 (Cross-correlation)** 将干声和湿声在时间轴上实现完美的绝对对齐。哪怕差 1 毫秒（44 个采样点），模型也会学废。
 3. **数据切片**：将对齐好的长音频切割成无数个小片段（Chunks，例如每段 0.5 秒），并划分为训练集 (80%)、验证集 (10%) 和测试集 (10%)。
- **阶段产出**：一套自制的、完美对齐的高质量 **.wav** 乐器配对数据集。
这 3 分钟里必须包含什么？

数据量的长度不重要，数据的“信息密度”最重要。干声（Input）必须尽可能触发音箱的所有物理反应。在 NAM 的官方标准中，一段标准的 3 分钟测试信号（**v3_0_0.wav**）包含了以下三个核心部分：

- **测试信号区 (Test Signals, 约 30 秒)**：
 - **正弦波扫频 (Sine Sweeps)**：从极低频（20Hz）滑到极高频（20kHz），用来让模型学习设备的 EQ（频率响应）曲线。
 - **白噪音/粉红噪音 (Noise Bursts)**：突然的噪音爆发，用来测试硬件的瞬间动态响应和瞬态（Transients）表现。
- **真实演奏区 (Real Playing, 约 2 分钟)**：
 - 极其干净的电吉他/贝斯 DI（Direct Inject）信号。
 - **动态覆盖**：必须有极轻的拨弦（测清音还原）和极重的狂扫（测过载/失真临界点）。
 - **频段覆盖**：必须有低音弦的闷音（Palm Mutes，非常考验模型低频的紧实度）和高把位的推弦、颤音。
 - **复音与单音**：既要有单音 Solo，也要有复杂的六根弦同时震动的和弦（Chords，考验模型的互调失真处理）。
- **静音区 (Silence, 约几秒)**：在开头和结尾留白，用来让模型学习目标硬件的底噪（Noise Floor）水平。

阶段三：TCN 架构与 Loss 函数复刻

目标：不依赖现有封装，在 PyTorch 中从零手写 NAM 的核心网络。

- 核心任务：
 1. 基础模块编写：手写因果卷积层（通过向一侧 Padding 实现），手写带有膨胀率（Dilation = 1, 2, 4, 8...）的一维卷积。
 2. 门控机制：实现 WaveNet 特有的 tanh 与 sigmoid 相乘的门控激活（Gated Activation）和残差连接（Residual Connection）。
 3. 损失函数实现：放弃传统的均方误差 (MSE)，亲手用代码实现 **ESR (Error Signal Ratio)** 和直流偏移损失 (DC Loss)。
- 阶段产出：一个包含 `model.py` (网络结构) 和 `loss.py` (损失函数) 的独立 Python 工程。

阶段四：模型训练与超参数消融实验

目标：炼丹与调优，探寻性能与音质的最佳平衡点。

- 核心任务：
 1. 编写训练循环：实现前向传播、反向传播、梯度裁剪 (Gradient Clipping) 以及学习率衰减策略。
 2. 记录与监控：接入 TensorBoard，实时观察 Train Loss 和 Validation Loss 的下降曲线，防止过拟合。
 3. 消融实验 (Ablation Study)：这是学术论文的核心。改变网络的“深度（层数）”和“宽度（通道数）”，对比不同体量模型在验证集上的 ESR 数值差异，找出性价比最高的参数组合。
- 阶段产出：多个训练好的 `.pt` 权重文件，以及 TensorBoard 的训练曲线对比图。

阶段五：离线推理与主客观评测

目标：虽然不做实时插件，但必须证明模型的实际效果。

- 核心任务：
 1. 编写离线推理脚本：写一个 `inference.py`，让用户可以输入一段从未见过的干声 `.wav`，通过模型计算，输出一段生成的湿声 `.wav`。
 2. 客观指标对线：将模型生成的音频与真实的湿声进行对比，计算测试集上的最终 ESR。
 3. 主观听音测试：组织一场盲听测试 (ABX Test)，让几位受试者戴上耳机，分辨哪段是真实的物理硬件声音，哪段是模型生成的声音，并收集统计数据。
- 阶段产出：一组用于展示的对比如音频 (Dry -> Real Wet -> AI Wet)，以及包含客观图表和主观评价的最终报告。

假设你使用 48kHz 的采样率，3 分钟的音频就包含了 **8,640,000 个采样点（数据对）**。对于一个参数量只有几十万到一两百万的轻量级 TCN/LSTM 网络来说，近千万次的前向传播足以让它收敛，学会输入输出之间的非线性映射规律。

3. 数据格式与规范 (Technical Specs)

- 格式： `.wav` 文件（不要用 `.mp3`，有损压缩会毁掉高频和瞬态）。
- 采样率： 推荐 **48 kHz** 或 44.1 kHz
- 位深： 推荐 **24-bit**，保留充足的动态范围。
- 声道： 单声道 (Mono)。我们建模的是单声道乐器/音箱，双声道会浪费一倍的计算资源。

必须做到 **100%** 的采样点级对齐